

ใบงานที่ 2

เรื่อง ทดลองวงจรถอดรหัสและวงจรเข้ารหัส

(Encoder and Decoder)

จัดทำโดย

นาย ศิริพงศ์ บุญประสิทธิ์

รหัส ENGCE200_SEC1

เสนอ

อาจารย์สมนึก สุระธง

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE200

การออกแบบดิจิทัล

(Digital Systems Design)

หลักสูตร วศ.บ.วิศวกรรมคอมพิวเตอร์

สาขาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาคพายัพ เชียงใหม่

ภาคเรียนที่ 1 ปีการศึกษา 2568

ใบงานที่ 2

ชื่อบทเรียน ทดลองวงจรถอดรหัสและวงจรเข้ารหัส

(Encoder and Decoder)

จุดประสงค์การสอน

ปฏิบัติการทดลองวงจรถอดรหัสและวงจรเข้ารหัส (Encoder and Decoder)

ออกแบบและทดสอบวงจรเข้ารหัส

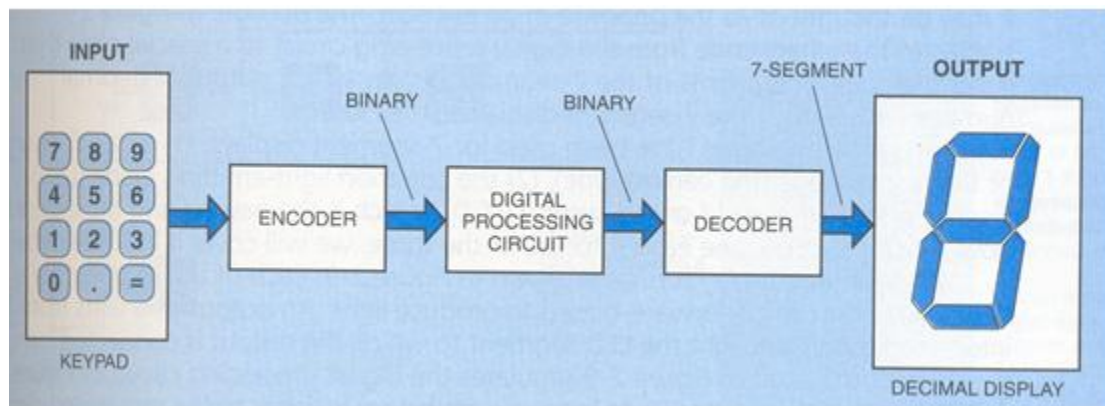
ออกแบบและทดสอบวงจรถอดรหัส

1. ทฤษฎี

วิธีการออกแบบวงจรเข้ารหัส

วิธีการออกแบบวงจรถอดรหัส

2. ให้ออกแบบวงจรถอดรหัสและวงจรเข้ารหัส (Encoder and Decoder) ดังรูปที่ 1



รูปที่ 1

ให้ทำวงจรสแกน Keypad ให้สามารถทำงานได้โดยการ กดคีย์ใดก็แสดงเลขนั้นที่ 7-segment

3. ผลการทดลอง

4. สรุปผลการทดลอง

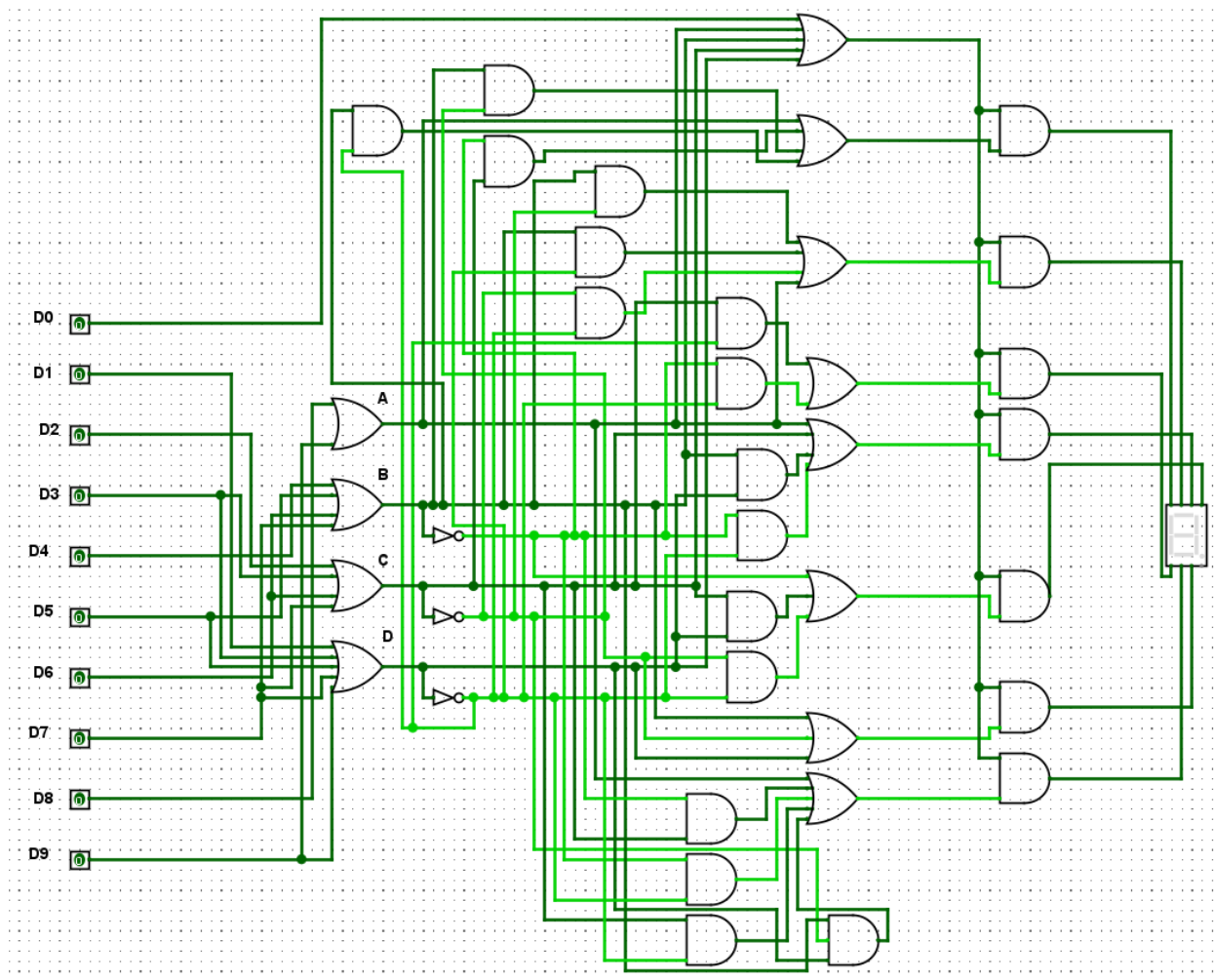
5. คำถามหลังการทดลอง

1. บอกหลักการทำงานของวงจรเข้ารหัส

2. บอกหลักการทำงานของวงจรถอดรหัส

3. ถ้าจะต่อให้แสดงตัวเลขเป็นสองหลักต้องทำอย่างไร

encoder-decoder-7segment.circ



encoder-decoder-7segment.vhdl

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity test is
```

```
    Port (
```

```
        D0 : in STD_LOGIC;
```

```
        D1 : in STD_LOGIC;
```

```
        D2 : in STD_LOGIC;
```

```
        D3 : in STD_LOGIC;
```

```
        D4 : in STD_LOGIC;
```

```
        D5 : in STD_LOGIC;
```

```
        D6 : in STD_LOGIC;
```

```
        D7 : in STD_LOGIC;
```

```
        D8 : in STD_LOGIC;
```

```
        D9 : in STD_LOGIC;
```

```
        seg_a : out STD_LOGIC;
```

```
        seg_b : out STD_LOGIC;
```

```
        seg_c : out STD_LOGIC;
```

```
        seg_d : out STD_LOGIC;
```

```
        seg_e : out STD_LOGIC;
```

```
        seg_f : out STD_LOGIC;
```

```
        seg_g : out STD_LOGIC
```

```
    );
```

```
end test;
```

```
architecture Behavioral of test is
```

```
    signal A,B,C,D : std_logic;
```

```
    signal NA,NB,NC,ND : std_logic;
```

```
begin
```

```
-----
```

```
-- Encoder
```

```
-----  
A <= D8 or D9;
```

```
B <= D4 or D5 or D6 or D7;
```

```
C <= D2 or D3 or D6 or D7;
```

```
D <= D1 or D3 or D5 or D7 or D9;
```

```
-----  
-- NOT Gate
```

```
-----  
NA <= not A;
```

```
NB <= not B;
```

```
NC <= not C;
```

```
ND <= not D;
```

```
-----  
-- Decoder
```

```
-----  
seg_a <= A or C or (B and D) or (NB and ND);
```

```
seg_b <= NB or (C and D) or (NC and ND);
```

```
seg_c <= B or NC or D;
```

```
seg_d <= A or (NB and C) or (NB and ND) or (C and ND) or (B and NC and  
D);
```

```
seg_e <= (NB and ND) or (C and ND);
```

```
seg_f <= A or (B and NC) or (C and ND) or (NC and ND);
```

```
seg_g <= A or (NB and C) or (B and NC) or (B and ND);
```

```
end Behavioral;
```

tb_encoder-decoder-7segment.vhdl

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity tb_test is

end tb_test;

architecture Behavioral of tb_test is

component test

Port (

D0 : in STD_LOGIC;

D1 : in STD_LOGIC;

D2 : in STD_LOGIC;

D3 : in STD_LOGIC;

D4 : in STD_LOGIC;

D5 : in STD_LOGIC;

D6 : in STD_LOGIC;

D7 : in STD_LOGIC;

D8 : in STD_LOGIC;

D9 : in STD_LOGIC;

seg_a : out STD_LOGIC;

seg_b : out STD_LOGIC;

seg_c : out STD_LOGIC;

seg_d : out STD_LOGIC;

seg_e : out STD_LOGIC;

seg_f : out STD_LOGIC;

seg_g : out STD_LOGIC

);

end component;

```
signal D0,D1,D2,D3,D4,D5,D6,D7,D8,D9 : STD_LOGIC := '0';
```

```
signal seg_a,seg_b,seg_c,  
        seg_d,seg_e,seg_f,seg_g : STD_LOGIC;
```

```
begin
```

```
UUT : test
```

```
port map(  
    D0=>D0, D1=>D1, D2=>D2, D3=>D3, D4=>D4,  
    D5=>D5, D6=>D6, D7=>D7, D8=>D8, D9=>D9,  
    seg_a=>seg_a,  
    seg_b=>seg_b,  
    seg_c=>seg_c,  
    seg_d=>seg_d,  
    seg_e=>seg_e,  
    seg_f=>seg_f,  
    seg_g=>seg_g  
);
```

```
stim_proc : process
```

```
begin
```

```
    D0<='1'; wait for 20 ns;  
    D0<='0';
```

```
    D1<='1'; wait for 20 ns;  
    D1<='0';
```

```
    D2<='1'; wait for 20 ns;  
    D2<='0';
```

```
D3<='1'; wait for 20 ns;  
D3<='0';
```

```
D4<='1'; wait for 20 ns;  
D4<='0';
```

```
D5<='1'; wait for 20 ns;  
D5<='0';
```

```
D6<='1'; wait for 20 ns;  
D6<='0';
```

```
D7<='1'; wait for 20 ns;  
D7<='0';
```

```
D8<='1'; wait for 20 ns;  
D8<='0';
```

```
D9<='1'; wait for 20 ns;  
D9<='0';
```

```
wait;
```

```
end process;
```

```
end Behavioral;
```